How to run

- **Data source (default):** GitHub Raw – https://raw.githubusercontent.com/thaiminhhuongdinh-crypto/credit-risk-demo/refs/heads/main/credit_risk_dataset.csv
- **No Google Drive / no `kaggle.json` needed for the default path.**

Steps

1. Open the Colab link → if it's "View only", click **Open in Playground**.
2. Go to **Runtime → Factory reset runtime** (clean start).
3. Click **Runtime → Run all**. The notebook auto-installs pinned libraries and auto-downloads data.

Optional: Use your own Google Drive CSV instead

If you prefer not to use the GitHub link, you can download the CSV and place it in **your** Drive, then run:

```
from google.colab import drive
drive.mount('/content/drive')

# Copy your CSV from Drive into the notebook's ./data/ folder
!mkdir -p data
!cp "/content/drive/MyDrive/credit_risk_dataset.csv" "data/credit_risk_dataset.csv"  # or: MyDrive/data/credit_risk_dataset.csv

# (Now just run the rest of the notebook)
```

```
!pip -q install numpy==1.26.4 pandas==2.2.2 scikit-learn==1.4.2 xgboost==2.0.3 joblib==1.3.2 kaggle==1.6.17
import sys, numpy as np, pandas as pd, sklearn, xgboost as xgb
print("Py", sys.version.split()[0], "| np", np.__version__, "| pd", pd.__version__, "| skl", sklearn.__version__, "| xgb", xgb.__ve
```

```
Py 3.12.11 | np 1.26.4 | pd 2.2.2 | skl 1.4.2 | xgb 2.0.3
```

```
# ==== SETUP ENVIRONMENT ====


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import (precision_score, recall_score, accuracy_score,
                             f1_score, roc_auc_score, average_precision_score,
                             confusion_matrix, ConfusionMatrixDisplay)
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
import warnings
warnings.filterwarnings("ignore")
```

```
# ==== ONE-CELL UNIVERSAL LOADER: Local -> URL -> Kaggle ====
!pip -q install gdown

import os, re, glob, sys, subprocess, requests, pandas as pd, hashlib

DATA_DIR  = "data"; os.makedirs(DATA_DIR, exist_ok=True)
DATA_FILE = "credit_risk_dataset.csv"
DATA_PATH = f"{DATA_DIR}/{DATA_FILE}"

#Link from github
DATA_URL = "https://raw.githubusercontent.com/thaiminhhuongdinh-crypto/credit-risk-demo/main/credit_risk_dataset.csv"


EXPECTED_SHA256 = ""
```

```
KAGGLE_DATASET = "laotse/credit-risk-dataset"  # fallback if need

def sha256_file(p):
    h = hashlib.sha256()
    with open(p,'rb') as f:
        for c in iter(lambda: f.read(1<<20), b''): h.update(c)
    return h.hexdigest()

def from_local():
    return os.path.exists(DATA_PATH)

def from_url():
    if not DATA_URL: return False
    url = DATA_URL.strip()

    #If it’s Google Drive -> use gdown (stable, bypass confirmation/quota)
    if "drive.google.com" in url:
        m = re.search(r"/d/([A-Za-z0-9_-]{10,})", url) or re.search(r"[?&]id=([A-Za-z0-9_-]{10,})", url)
        if not m:
            print("❌ Không trích được FILE_ID từ DATA_URL Drive."); return False
        file_id = m.group(1)
        try:
            import gdown  # noqa
        except Exception:
            subprocess.run([sys.executable, "-m", "pip", "install", "-q", "gdown"], check=True)
            import gdown
        gdown.download(f"https://drive.google.com/uc?id={file_id}", DATA_PATH, quiet=False)
        return os.path.exists(DATA_PATH)

    # URL from(GitHub raw, S3, Dropbox…)
    r = requests.get(url, timeout=60, allow_redirects=True)
    r.raise_for_status()
    open(DATA_PATH, "wb").write(r.content)
    return True

def from_kaggle():
    kj = os.path.expanduser("~/.kaggle/kaggle.json")
    if not os.path.exists(kj):
        print("⚠️ Không thấy ~/.kaggle/kaggle.json — bỏ qua Kaggle.")
        return False
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "kaggle"], check=True)
    from kaggle.api.kaggle_api_extended import KaggleApi
    api = KaggleApi(); api.authenticate()
    api.dataset_download_files(KAGGLE_DATASET, path=DATA_DIR, unzip=True)
    cands = [p for p in glob.glob(f"{DATA_DIR}/*.csv")]
    if cands and not os.path.exists(DATA_PATH):
        os.replace(cands[0], DATA_PATH)
    return os.path.exists(DATA_PATH)

ok = from_local() or from_url() or from_kaggle()
if not ok:
    raise SystemExit("No data available. Place the CSV in ./data/, or fill in DATA_URL, or upload kaggle.json and then Run all.")

if EXPECTED_SHA256:
    cur = sha256_file(DATA_PATH); print("SHA256:", cur)
    assert cur == EXPECTED_SHA256, "Hash mismatch archived"

df = pd.read_csv(DATA_PATH)
print("✅ Loaded:", df.shape, "| CSV:", DATA_PATH)
```

```
✅ Loaded: (32581, 12) | CSV: data/credit_risk_dataset.csv
```

```
import requests, pandas as pd
print(requests.head(DATA_URL).status_code)
df = pd.read_csv(DATA_URL)
print(df.shape)
```

```
200
(32581, 12)
```

```
print(df.shape)
```

```
TARGET = "loan_status"
y = df[TARGET].astype(int)
X = df.drop(columns=[TARGET])
```

```
(32581, 12)
```

```
df.head()
```

	person_age	person_income	person_home_ownership	person_emp_length	loan_intent	loan_grade	loan_amnt	loan_int_rate	loan_sta
0	22	59000	RENT	123.0	PERSONAL	D	35000	16.02	
1	21	9600	OWN	5.0	EDUCATION	B	1000	11.14	
2	25	9600	MORTGAGE	1.0	MEDICAL	C	5500	12.87	
3	23	65500	RENT	4.0	MEDICAL	C	35000	15.23	
4	24	54400	RENT	8.0	MEDICAL	C	35000	14.27	

Next steps: Generate code with df | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32581 entries, 0 to 32580
Data columns (total 12 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   person_age                  32581 non-null  int64
 1   person_income               32581 non-null  int64
 2   person_home_ownership       32581 non-null  object
 3   person_emp_length           31686 non-null  float64
 4   loan_intent                 32581 non-null  object
 5   loan_grade                  32581 non-null  object
 6   loan_amnt                   32581 non-null  int64
 7   loan_int_rate               29465 non-null  float64
 8   loan_status                 32581 non-null  int64
 9   loan_percent_income         32581 non-null  float64
 10  cb_person_default_on_file   32581 non-null  object
 11  cb_person_cred_hist_length  32581 non-null  int64
dtypes: float64(3), int64(5), object(4)
memory usage: 3.0+ MB
```

Data Type Standardization

To ensure consistent and correct data types for modeling:

- String columns (`loan_grade`, `loan_intent`, etc.) were stripped and capitalized, then cast to categorical.
- Binary flag `cb_person_default_on_file` was mapped from "Y"/"N" to 1/0.
- `loan_grade` was treated as an ordinal variable with preserved order (A < B < ... < G).
- All numeric columns were explicitly cast using `pd.to_numeric()` with error coercion for robustness.

```
# ==== STANDARDIZE DATA TYPES ====

# 1. Normalize string columns (remove spaces, unify casing)
for col in ["person_home_ownership", "loan_intent", "loan_grade", "cb_person_default_on_file"]:
    df[col] = df[col].astype(str).str.strip().str.upper()

# 2. Convert to categorical dtype
cat_cols = ["person_home_ownership", "loan_intent", "loan_grade", "cb_person_default_on_file"]
df[cat_cols] = df[cat_cols].astype("category")

# 3. Map Y/N → 1/0 using nullable integer type (allows NA values)
df["cb_person_default_on_file"] = (
    df["cb_person_default_on_file"]
    .map({"N": 0, "Y": 1})
    .astype("Int8")  # capital I: allows missing (nullable)
)

# 4. Set ordinal order for loan_grade (A < B < C < ...)
grade_order = [g for g in "ABCDEFG" if g in df["loan_grade"].cat.categories]
```

```
df["loan_grade"] = df["loan_grade"].cat.set_categories(grade_order, ordered=True)

# 5. Ensure numeric columns are numeric (auto convert + safeguard)
num_cols = [
    "person_age", "person_income", "person_emp_length", "loan_amnt",
    "loan_int_rate", "loan_percent_income", "cb_person_cred_hist_length", "loan_status"
]
df[num_cols] = df[num_cols].apply(pd.to_numeric, errors="coerce")

#  Check final dtypes
print(df.dtypes)
```

```
person_age                     int64
person_income                  int64
person_home_ownership       category
person_emp_length            float64
loan_intent                 category
loan_grade                  category
loan_amnt                      int64
loan_int_rate                float64
loan_status                    int64
loan_percent_income          float64
cb_person_default_on_file      Int8
cb_person_cred_hist_length     int64
dtype: object
```

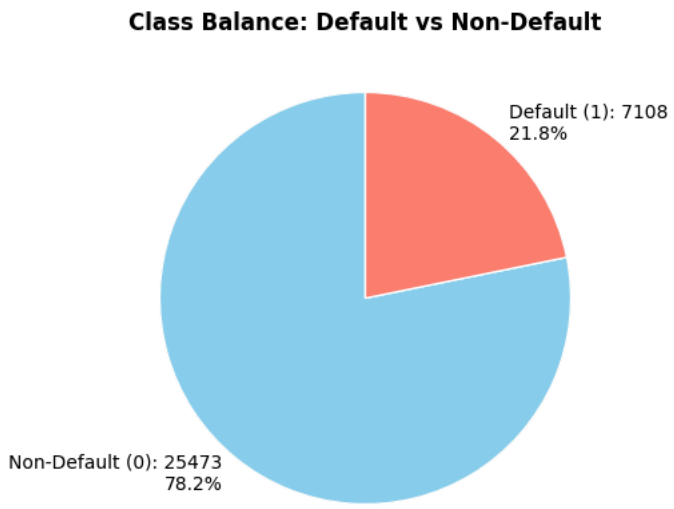
```
df.describe()
```

	person_age	person_income	person_emp_length	loan_amnt	loan_int_rate	loan_status	loan_percent_income	cb_person_defa
count	32581.000000	3.258100e+04	31686.000000	32581.000000	29465.000000	32581.000000	32581.000000	
mean	27.734600	6.607485e+04	4.789686	9589.371106	11.011695	0.218164	0.170203	
std	6.348078	6.198312e+04	4.142630	6322.086646	3.240459	0.413006	0.106782	
min	20.000000	4.000000e+03	0.000000	500.000000	5.420000	0.000000	0.000000	
25%	23.000000	3.850000e+04	2.000000	5000.000000	7.900000	0.000000	0.090000	
50%	26.000000	5.500000e+04	4.000000	8000.000000	10.990000	0.000000	0.150000	
75%	30.000000	7.920000e+04	7.000000	12200.000000	13.470000	0.000000	0.230000	
max	144.000000	6.000000e+06	123.000000	35000.000000	23.220000	1.000000	0.830000	

Class balance

```
import matplotlib.pyplot as plt

# Define the target column (0 = non-default, 1 = default)
target_col = "loan_status"

# Count the number of instances for each class (0 and 1)
counts = df[target_col].value_counts().sort_index()  # ensures order: 0 then 1

# Create human-readable labels (e.g., Default (1): 7108)
labels = [f"Non-Default (0): {counts[0]}", f"Default (1): {counts[1]}"]

# Compute percentage share of each class
pct = counts / counts.sum() * 100  # in percentage

# Draw pie chart to visualize class imbalance
fig, ax = plt.subplots(figsize=(5, 5))
ax.pie(
    counts,
    labels=[f"{lab}\n{p:.1f}%" for lab, p in zip(labels, pct)],
    autopct=None,
    startangle=90,  # rotate to start from top
    colors=["skyblue", "salmon"],
    wedgeprops={"edgecolor": "white"}  # makes slices look nicer
)

ax.set_title("Class Balance: Default vs Non-Default", fontsize=12, fontweight="bold")
```

```
plt.show()
```



Check missing values

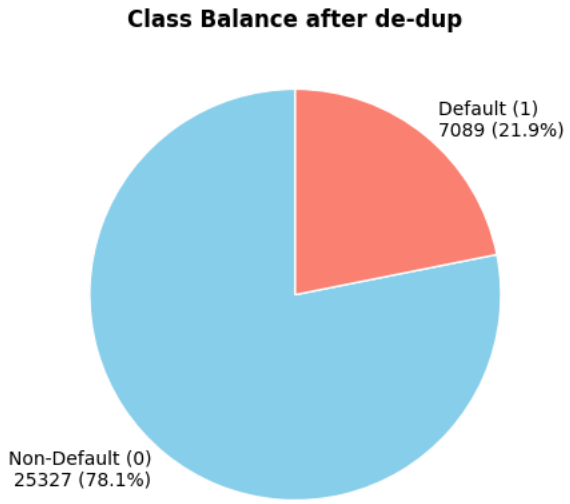
```
def missing_report(df: pd.DataFrame):
    cnt = df.isna().sum()
    pct = (cnt/len(df)*100).round(2)
    rep = pd.DataFrame({'missing_count': cnt, 'missing_pct': pct}).sort_values('missing_count', ascending=False)
    return rep[rep.missing_count > 0]

missing_report(df)
```

	missing_count	missing_pct
loan_int_rate	3116	9.56
person_emp_length	895	2.75

Drop duplicate and check again class balance

```
# 1) Count duplicates
dup_count = df.duplicated(keep=False).sum()
print(f"Total duplicated rows (exact duplicates): {dup_count}")

# 2) Drop duplicates (keep the first occurrence)
before = df.shape
df = df.drop_duplicates(keep='first')
after = df.shape
print(f"Dropped {before[0]-after[0]} rows. New shape: {after}")
print(df.shape)
# 3) Pie chart for class balance
import matplotlib.pyplot as plt

target_col = "loan_status"
counts = df[target_col].value_counts().sort_index()
pct = counts / counts.sum() * 100

fig, ax = plt.subplots(figsize=(5, 5))
labels = [f"Non-Default (0)\n{counts[0]} ({pct[0]:.1f}%)",
          f"Default (1)\n{counts[1]} ({pct[1]:.1f}%)"]
ax.pie(counts, labels=labels, startangle=90,
       colors=["skyblue","salmon"], wedgeprops={'edgecolor':'white'})
ax.set_title("Class Balance after de-dup", fontweight="bold")
plt.show()
```

```
Total duplicated rows (exact duplicates): 330
Dropped 165 rows. New shape: (32416, 12)
(32416, 12)
```

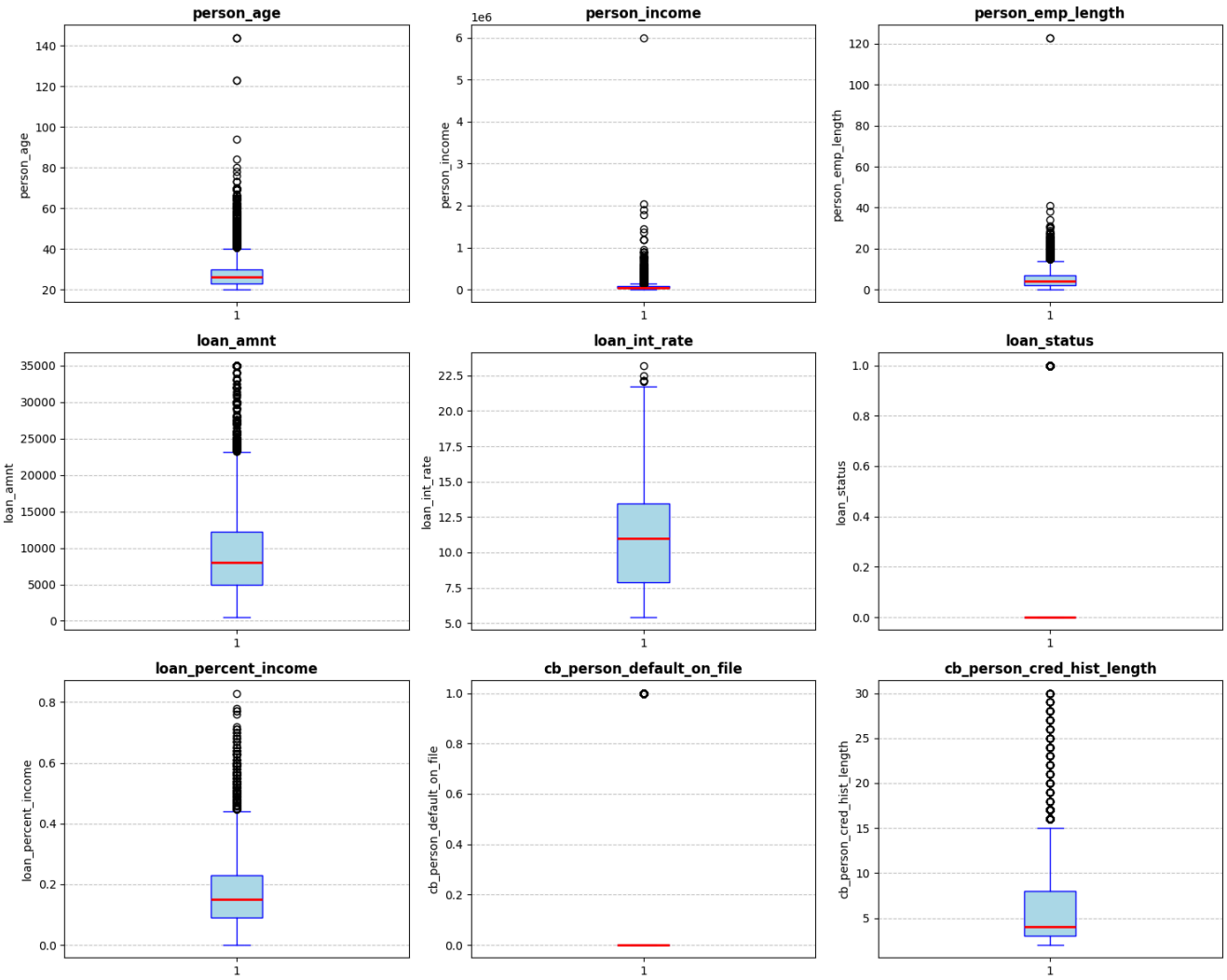


Check outliers

```
import numpy as np
import pandas as pd

def iqr_outlier_columns(df: pd.DataFrame, k: float = 1.5):
    """
    Identify columns with outliers using the IQR rule.
    Returns:
        - List of columns that contain outliers
        - Dictionary with outlier count per column
    """
    num = df.select_dtypes(include="number")  # select only numeric columns
    cols_with_outlier = []
    counts = {}

    for c in num.columns:
        s = num[c].dropna()  # drop NaN to avoid errors

        if s.empty:
            continue

        # Compute IQR bounds
        q1, q3 = s.quantile(0.25), s.quantile(0.75)
        iqr = q3 - q1
        lo, hi = q1 - k * iqr, q3 + k * iqr

        # Detect outliers
        mask = (s < lo) | (s > hi)
        cnt = int(mask.sum())

        if cnt > 0:
            cols_with_outlier.append(c)
            counts[c] = cnt

    return cols_with_outlier, counts
    # Run the outlier detection function
cols, counts = iqr_outlier_columns(df)

# Summary output
print("Are there any outliers?", bool(cols))
print("Columns with outliers:", cols)
print("Number of outliers per column (top 10):", sorted(counts.items(), key=lambda x: -x[1])[:10])
```

```
Are there any outliers? True
Columns with outliers: ['person_age', 'person_income', 'person_emp_length', 'loan_amnt', 'loan_int_rate', 'loan_status', 'loan_perc
Number of outliers per column (top 10): [('loan_status', 7089), ('cb_person_default_on_file', 5730), ('loan_amnt', 1679), ('person_
```

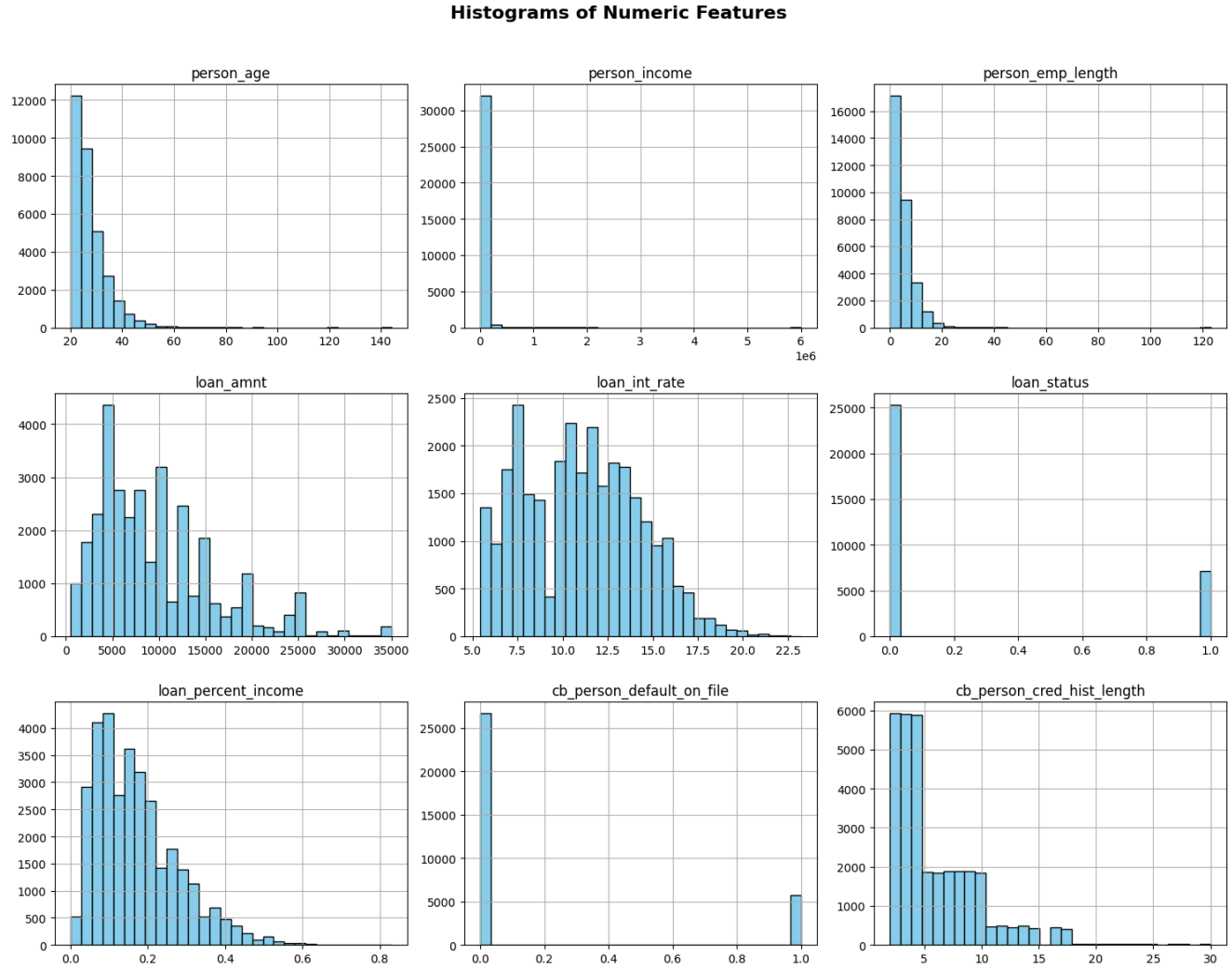
```
import math
import matplotlib.pyplot as plt

# Define the list of columns that contain outliers
n = len(cols)          # Total number of columns with outliers
ncols = 3              # Set number of columns per row for the subplots
nrows = math.ceil(n / ncols)  # Calculate the number of rows needed

# Create a grid of subplots with proper size
fig, axes = plt.subplots(nrows, ncols, figsize=(5*ncols, 4*nrows))  # each subplot is large enough

# Draw one boxplot per column
for ax, c in zip(axes.flatten(), cols):
    ax.boxplot(df[c].dropna(),          # Remove missing values before plotting
               vert=True,               # Vertical boxplot
               patch_artist=True,       # Fill the box with color
               boxprops=dict(facecolor="lightblue", color="blue"),       # Blue edge, light blue box
               medianprops=dict(color="red", linewidth=2),               # Red thick median line
               whiskerprops=dict(color="blue"),                          # Blue whiskers
               capprops=dict(color="blue"))                              # Blue caps

    ax.set_title(f"{c}", fontsize=12, fontweight="bold")                # Set the title of each subplot
    ax.grid(axis="y", linestyle="--", alpha=0.7)                        # Add horizontal gridlines
    ax.set_ylabel(c)                                                    # Label y-axis

# Remove any unused (empty) subplots if there are fewer columns than subplot slots
for ax in axes.flatten()[len(cols):]:
    ax.remove()

# Adjust spacing between subplots
plt.tight_layout()
plt.show()
```



EDA basic

```
import matplotlib.pyplot as plt

# Take all numeric columns
num_cols = df.select_dtypes(include='number').columns

# Draw histograms for all numeric columns.
df[num_cols].hist(bins=30, figsize=(15, 12), color="skyblue", edgecolor="black")
plt.suptitle("Histograms of Numeric Features", fontsize=16, fontweight="bold")
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



Split train/test

```
from sklearn.model_selection import train_test_split

TARGET = "loan_status"
X = df.drop(columns=TARGET)
y = df[TARGET]
```

```
# 1. Split 20% test
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

# 2. Split 20% of remaining 80% = 16% for validation
X_train, X_val, y_train, y_val = train_test_split(X_rest, y_rest, test_size=0.2, stratify=y_rest, random_state=42)

# wrap into dataframes for convenience
df_train = X_train.copy(); df_train[TARGET] = y_train
df_val = X_val.copy(); df_val[TARGET] = y_val
df_test = X_test.copy(); df_test[TARGET] = y_test

# Class balance check
def show_balance(y):
    p = y.value_counts(normalize=True).round(3) * 100
    return f"0: {p[0]}% | 1: {p[1]}%"

print(f"Train: {df_train.shape}, Balance: {show_balance(df_train[TARGET])}")
print(f"Val:   {df_val.shape}, Balance: {show_balance(df_val[TARGET])}")
print(f"Test:  {df_test.shape}, Balance: {show_balance(df_test[TARGET])}")
```

```
Train: (20745, 12), Balance: 0: 78.10000000000001% | 1: 21.9%
Val:   (5187, 12), Balance: 0: 78.10000000000001% | 1: 21.9%
Test:  (6484, 12), Balance: 0: 78.10000000000001% | 1: 21.9%
```

Logistic Regression (Baseline)

Step 1: Clipping (Outlier Handling)

Clip values into a reasonable range.

Reduce the impact of extreme outliers on the model.

```
def domain_clip(df):
    df = df.copy()
    df["person_age"] = df["person_age"].clip(18, 95)
    df["loan_int_rate"] = df["loan_int_rate"].clip(0, 45)
    df["person_emp_length"] = df["person_emp_length"].clip(0, 50)
    df["loan_percent_income"] = df["loan_percent_income"].clip(0, 1.5)
    return df

X_train_sc = domain_clip(X_train)
X_val_sc = domain_clip(X_val)
X_test_sc = domain_clip(X_test)
```

Step 2: Define Columns to Process

num_cols: numerical features that need imputation and scaling.

cat_cols: categorical features for One-Hot Encoding.

BINARY_COLS: binary features passed through as-is.

```
TARGET = "loan_status"
BINARY_COLS = ["cb_person_default_on_file"]

num_cols = X_train_sc.select_dtypes(include='number').columns.difference([TARGET] + BINARY_COLS).tolist()
cat_cols = [c for c in X_train_sc.columns if c not in num_cols + BINARY_COLS + [TARGET]]
```

Step 3: Create the ColumnTransformer

This defines preprocessing steps for each column type.

Numeric: median imputation + robust scaling.

Categorical: mode imputation + One-Hot Encoding.

Binary: passthrough.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, RobustScaler

def make_ohe():
    return OneHotEncoder(handle_unknown="ignore", sparse_output=False)

pre_lr = ColumnTransformer([
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", RobustScaler())
    ]), num_cols),

    ("bin", "passthrough", BINARY_COLS),

    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("ohe", make_ohe())
    ]), cat_cols)
], remainder="drop")
```

Step 4: Build the Pipeline with Logistic Regression

Combines preprocessing and modeling into one clean pipeline.

class_weight="balanced" helps handle class imbalance.

```
from sklearn.linear_model import LogisticRegression

pipe_lr = Pipeline([
    ("prep", pre_lr),
    ("model", LogisticRegression(
        solver="lbfgs", class_weight="balanced", max_iter=1000, random_state=42
    ))
])
```

Step 5:Hyperparameter Tuning (C)

Finds the best regularization strength C using cross-validation.

Chooses model with best average precision.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

param_grid = {"model__C": [0.1, 0.5, 1, 2, 5, 10]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

gs = GridSearchCV(pipe_lr, param_grid, scoring="average_precision", cv=cv, n_jobs=-1, refit=True)
gs.fit(X_train_sc, y_train)

best_lr = gs.best_estimator_
```

Step 6: Select Optimal Threshold (Cost-Based)

Iterates over thresholds from 0.01 to 0.99.

Chooses the one that minimizes business cost.

FN is weighted more heavily than FP (cost ratio 5:1).

```
from sklearn.metrics import confusion_matrix, precision_recall_fscore_support
import numpy as np

def pick_best_threshold(y_true, y_score, fn_cost=5.0, fp_cost=1.0):
    ths = np.linspace(0.01, 0.99, 99)
    best = None
    for t in ths:
        pred = (y_score >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
        cost = fn * fn_cost + fp * fp_cost
        prec, rec, f1, _ = precision_recall_fscore_support(y_true, pred, average="binary")
        if best is None or cost < best[0]:
            best = (cost, t, prec, rec, f1)
    return {"cost": best[0], "t": best[1], "f1": best[4]}
```

Step 7: Evaluate on Validation + Test

Applies the selected threshold on validation and test sets.

Evaluates ROC-AUC, PR-AUC, F1 score, and total cost.

```
from sklearn.metrics import roc_auc_score, average_precision_score

val_proba = best_lr.predict_proba(X_val_sc)[:, 1]
val_stats = pick_best_threshold(y_val, val_proba, fn_cost=5, fp_cost=1)

print(f"[VAL] LR | ROC-AUC={roc_auc_score(y_val, val_proba):.4f} | PR-AUC={average_precision_score(y_val, val_proba):.4f} "
      f"| F1@best={val_stats['f1']:.4f} | th={val_stats['t']:.2f} | Cost={val_stats['cost']:.0f}")

test_proba = best_lr.predict_proba(X_test_sc)[:, 1]
test_pred = (test_proba >= val_stats['t']).astype(int)

tn, fp, fn, tp = confusion_matrix(y_test, test_pred).ravel()
print(f"[TEST] LR | ROC-AUC={roc_auc_score(y_test, test_proba):.4f} | PR-AUC={average_precision_score(y_test, test_proba):.4f} "
      f"| F1@th={precision_recall_fscore_support(y_test, test_pred, average='binary')[2]:.4f} | Cost={5*fn + 1*fp}")
```

```
[VAL] LR | ROC-AUC=0.8741 | PR-AUC=0.7179 | F1@best=0.6540 | th=0.51 | Cost=1906
[TEST] LR | ROC-AUC=0.8696 | PR-AUC=0.7194 | F1@th=0.6476 | Cost=2507
```

Analysis of Model Tuning Results (Hyperparameter C).

```
cv_results = gs.cv_results_
mean_scores = cv_results["mean_test_score"]  # Average score of each C value
std_scores = cv_results["std_test_score"]    # Standard deviation between the folds
import numpy as np

n_folds = gs.cv.get_n_splits()  # Quantity of fold in cross-validation (5)
ci95 = 1.96 * std_scores / np.sqrt(n_folds)  # Standard CI formula


# Print information about each value C
for C, mean, std, ci in zip(cv_results["param_model__C"], mean_scores, std_scores, ci95):
    print(f"C={C}: CV mean = {mean:.4f} ± {std:.4f} | 95% CI = ±{ci:.4f}")
```

```
C=0.1: CV mean = 0.7147 ± 0.0078 | 95% CI = ±0.0068
C=0.5: CV mean = 0.7161 ± 0.0078 | 95% CI = ±0.0069
C=1: CV mean = 0.7164 ± 0.0079 | 95% CI = ±0.0069
C=2: CV mean = 0.7168 ± 0.0080 | 95% CI = ±0.0070
C=5: CV mean = 0.7170 ± 0.0080 | 95% CI = ±0.0070
C=10: CV mean = 0.7172 ± 0.0081 | 95% CI = ±0.0071
```

Mini-ablation for LR: L2 vs L1 (Appendix)

```
# Create Train + evaluate Logistic Regression with penalty “l1” or “l2”.
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import roc_auc_score, average_precision_score

def fit_eval_lr(penalty: str):
    # Choose the appropriate solver
    solver = "liblinear" if penalty == "l1" else "lbfgs"
    # Similar to the previous step
    # Create pipeline
    pipe = Pipeline([
        ("prep", pre_lr),
        ("model", LogisticRegression(
            penalty=penalty,
            solver=solver,
            class_weight="balanced",
            max_iter=1000,
            random_state=42
        ))
    ])

    # Grid search to find the optimal C according to PR-AUC
    param_grid = {"model__C": [0.1, 0.5, 1, 2, 5, 10]}
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    gs = GridSearchCV(pipe, param_grid, scoring="average_precision", cv=cv, n_jobs=-1, refit=True)
    gs.fit(X_train_sc, y_train)
    best_model = gs.best_estimator_

    # Validation
    val_proba = best_model.predict_proba(X_val_sc)[:, 1]
    val_stats = pick_best_threshold(y_val, val_proba, fn_cost=5.0, fp_cost=1.0)
    t_star = val_stats["t"]

    # Test
    test_proba = best_model.predict_proba(X_test_sc)[:, 1]
    test_pred = (test_proba >= t_star).astype(int)
    prec, rec, f1, _ = precision_recall_fscore_support(y_test, test_pred, average="binary")
    roc = roc_auc_score(y_test, test_proba)
    pr = average_precision_score(y_test, test_proba)

    return {
        "penalty": penalty.upper(),
        "C*": gs.best_params_["model__C"],
        "CV PR-AUC": gs.best_score_,
        "VAL ROC-AUC": roc_auc_score(y_val, val_proba),
        "VAL PR-AUC": average_precision_score(y_val, val_proba),
        "F1@best (VAL)": val_stats["f1"],
        "th*": t_star,
        "Cost (VAL)": val_stats["cost"],
        "TEST ROC-AUC": roc,
        "TEST PR-AUC": pr,
        "F1@th (TEST)": f1,
        "Cost@th": 5.0 * sum((y_test == 1) & (test_pred == 0)) + 1.0 * sum((y_test == 0) & (test_pred == 1))
    }
```

```
res_l2 = fit_eval_lr("l2")
res_l1 = fit_eval_lr("l1")
pd.DataFrame([res_l2, res_l1])
```

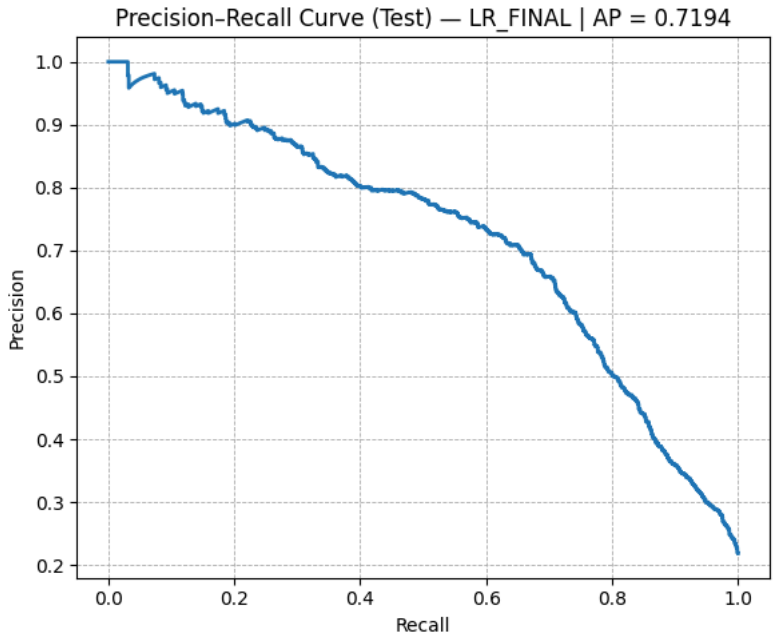
	penalty	C*	CV PR-AUC	VAL ROC-AUC	VAL PR-AUC	F1@best (VAL)	th*	Cost (VAL)	TEST ROC-AUC	TEST PR-AUC	F1@th (TEST)	Cost@th
0	L2	10	0.717227	0.874143	0.717945	0.653987	0.51	1906.0	0.869566	0.719391	0.647602	2507.0
1	L1	10	0.717468	0.874150	0.717002	0.657312	0.52	1904.0	0.869569	0.719550	0.649351	2533.0

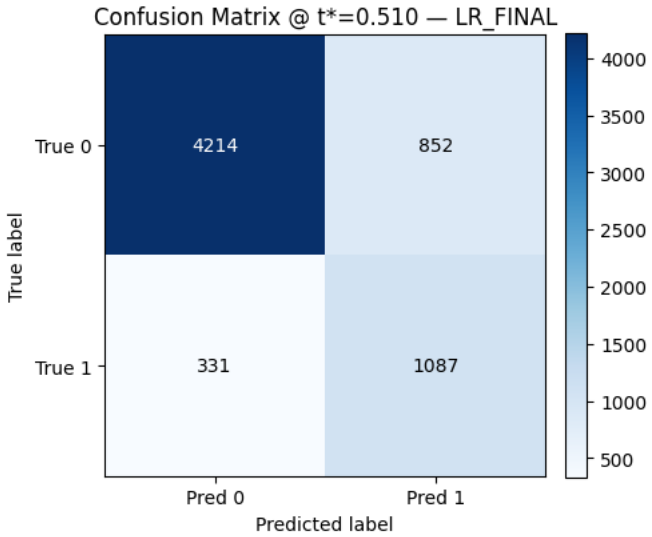
Draw PR curve + Confusion @ t*

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
```

```
    precision_recall_curve, average_precision_score,
    confusion_matrix, precision_score, recall_score, f1_score
)

def plot_pr_and_cm(y_test, test_proba, t_star, model_name="LR_FINAL", save=True):
    # --- PR curve (Test) ---
    ap = average_precision_score(y_test, test_proba)
    prec, rec, _ = precision_recall_curve(y_test, test_proba)

    plt.figure(figsize=(6,5))
    plt.plot(rec, prec, linewidth=2)
    plt.xlabel("Recall"); plt.ylabel("Precision")
    plt.title(f"Precision–Recall Curve (Test) — {model_name} | AP = {ap:.4f}")
    plt.grid(True, linestyle="--", linewidth=0.6)
    plt.tight_layout()
    if save: plt.savefig(f"pr_curve_test_{model_name}.png", dpi=300)
    plt.show()

    # --- Confusion matrix @ t* ---
    y_pred = (test_proba >= t_star).astype(int)
    cm = confusion_matrix(y_test, y_pred, labels=[0,1])

    fig, ax = plt.subplots(figsize=(5,4.5))
    im = ax.imshow(cm, interpolation='nearest', cmap="Blues")
    ax.figure.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

    ax.set(xticks=[0,1], yticks=[0,1],
           xticklabels=['Pred 0','Pred 1'], yticklabels=['True 0','True 1'],
           ylabel='True label', xlabel='Predicted label',
           title=f'Confusion Matrix @ t*={t_star:.3f} — {model_name}')
    thresh = cm.max()/2.0
    for i in range(2):
        for j in range(2):
            ax.text(j, i, int(cm[i, j]),
                    ha="center", va="center",
                    color="white" if cm[i, j] > thresh else "black")
    plt.tight_layout()
    if save: plt.savefig(f"confusion_matrix_test_{model_name}.png", dpi=300)
    plt.show()


    prec_t = precision_score(y_test, y_pred, zero_division=0)
    rec_t  = recall_score(y_test, y_pred, zero_division=0)
    f1_t   = f1_score(y_test, y_pred, zero_division=0)
    print(f"[{model_name}] AP(Test)={ap:.4f} | Precision={prec_t:.4f} | Recall={rec_t:.4f} | F1={f1_t:.4f} | t*={t_star:.3f}")
```

```
# Threshold has been CHOSEN on Validation
t_star_lr = float(val_stats["t"])

# Draw PR curve + Confusion matrix @ t* for LR_FINA
plot_pr_and_cm(y_test, test_proba, t_star_lr, model_name="LR_FINAL", save=True)
```






```
[LR_FINAL] AP(Test)=0.7194 | Precision=0.5606 | Recall=0.7666 | F1=0.6476 | t*=0.510
```

Cost scenario (Appendix)

```
import numpy as np, pandas as pd
from sklearn.metrics import confusion_matrix, f1_score, precision_recall_fscore_support

def choose_threshold_by_cost(y_true, y_score, cfn, cfp):
    ts = np.linspace(0.01, 0.99, 99)
    best_t, best_cost, best_stats = 0.5, float("inf"), None
    for t in ts:
        pred = (y_score >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
        cost = cfn*fn + cfp*fp
        prec, rec, f1, _ = precision_recall_fscore_support(y_true, pred, average="binary", zero_division=0)
        if cost < best_cost:
            best_t, best_cost, best_stats = t, cost, (prec, rec, f1)
    return best_t, best_cost, best_stats

# 1) cost scenarios

scenarios = [("FN:FP=1:1", 1,1), ("3:1",3,1), ("5:1",5,1), ("7:1",7,1)]

rows = []
for name, cfn, cfp in scenarios:
```

```
    t, val_cost, (p,r,f1) = choose_threshold_by_cost(y_val, val_proba, cfn, cfp)
    # áp t lên TEST
    test_pred = (test_proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, test_pred).ravel()
    test_cost = cfn*fn + cfp*fp
    rows.append([name, round(t,2), round(p,3), round(r,3), round(f1,3), int(val_cost), int(test_cost)])

# 2) Add F1-based row
def choose_threshold_by_f1(y_true, y_score):
    ts = np.linspace(0.01, 0.99, 99)
    f1s = [f1_score(y_true, (y_score>=t).astype(int)) for t in ts]
    i = int(np.argmax(f1s))
    return float(ts[i]), float(f1s[i])

t_f1, f1_val = choose_threshold_by_f1(y_val, val_proba)
test_pred_f1 = (test_proba >= t_f1).astype(int)
prec, rec, f1_test, _ = precision_recall_fscore_support(y_test, test_pred_f1, average="binary", zero_division=0)
rows.append(["F1-based", round(t_f1,2), None, None, round(f1_val,3), None, None])

appendix_tbl = pd.DataFrame(rows, columns=["Scenario","t* (VAL)","P_VAL","R_VAL","F1_VAL","Cost_VAL","Cost_TEST"])
print(appendix_tbl)
```

```
    Scenario  t* (VAL)  P_VAL  R_VAL  F1_VAL  Cost_VAL  Cost_TEST
0  FN:FP=1:1      0.76  0.739  0.597   0.660     696.0      881.0
1        3:1      0.56  0.599  0.765   0.672    1382.0     1797.0
2        5:1      0.51  0.559  0.788   0.654    1906.0     2507.0
3        7:1      0.33  0.417  0.876   0.565    2374.0     3073.0
4   F1-based      0.64    NaN    NaN   0.682       NaN        NaN
```

XGBoost

Step 1: Fit & Transform the Data Using pre (Pipeline)

pre is a ColumnTransformer (e.g., encoding + scaling).

Must fit it only on the training data and transform the train/val/test splits consistently.

```
pre = pre_lr
pre.fit(X_train, y_train)         # Train the preprocessing pipeline on training data
Xtr = pre.transform(X_train)
Xva = pre.transform(X_val)
Xte = pre.transform(X_test)
```

Step 2: Compute scale_pos_weight to Handle Class Imbalance

When the positive class is significantly underrepresented, scale_pos_weight helps XGBoost give it more attention.

This value is passed to the model as a hyperparameter.

```
pos = int((y_train == 1).sum())  # Count of positive class
neg = int((y_train == 0).sum())  # Count of negative class
spw = max(1.0, neg / max(pos, 1))  # Class balancing ratio (avoid division by 0)
```

Step 3: Create DMatrix (XGBoost's Optimized Data Format)

xgb.DMatrix is a highly optimized data structure required for xgb.train().

You must create a DMatrix separately for training, validation, and testing.

```
import xgboost as xgb

dtr = xgb.DMatrix(Xtr, label=y_train)
dva = xgb.DMatrix(Xva, label=y_val)
```

```
dte = xgb.DMatrix(Xte, label=y_test)
```

Step 4: Define Hyperparameters & Train with Early Stopping

Early stopping prevents overfitting and saves computation time.

The best iteration is automatically tracked using the PR-AUC score on the validation set.

```
params = {
    "objective": "binary:logistic",   # Binary classification
    "eval_metric": "aucpr",           # PR-AUC metric (better for imbalanced data)
    "eta": 0.05,                      # Learning rate
    "max_depth": 6,
    "subsample": 0.8,
    "colsample_bytree": 0.8,
    "lambda": 1.0,                    # L2 regularization
    "scale_pos_weight": spw,
    "tree_method": "hist",            # Use 'hist' method for speed
    "seed": 42
}
num_boost_round = 2000
bst = xgb.train(
    params,
    dtr,
    num_boost_round=num_boost_round,
    evals=[(dva, "val")],
    early_stopping_rounds=50,        # Stop if no improvement after 50 rounds
    verbose_eval=False
)
```

Step 5: Retrieve the Best Iteration

```
best_ntree = getattr(bst, "best_ntree_limit", None)
```

Step 6: Choose Optimal Threshold t* for F1 Score on Validation Set

Since the model outputs probabilities, we need to find the optimal cutoff t* that maximizes F1 on the validation set.

This is essential for imbalanced classification tasks.

```
import numpy as np
from sklearn.metrics import f1_score

def predict_proba(bst, dmat):
    if best_ntree is not None:
        return bst.predict(dmat, ntree_limit=best_ntree)
    else:
        return bst.predict(dmat)

proba_val  = predict_proba(bst, dva)
proba_test = predict_proba(bst, dte)

ths = np.linspace(0.01, 0.99, 99)  # Threshold candidates
t_star_xgb = ths[np.argmax([f1_score(y_val, (proba_val >= t).astype(int)) for t in ths])]
```

Step 7: Evaluate on the Test Set Using t*

```
# Predict probabilities for validation and test sets
proba_val = predict_proba(bst, dva)
proba_test = predict_proba(bst, dte)

# Predict classes using optimal threshold t*
```

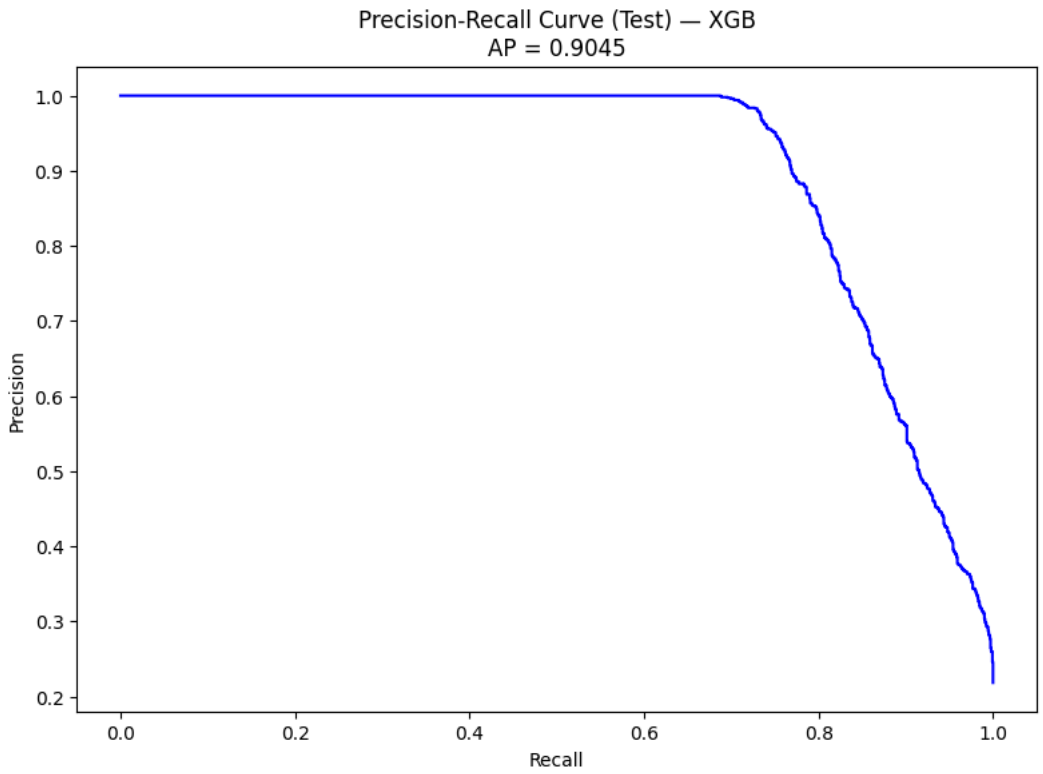
```
pred_val  = (proba_val  >= t_star_xgb).astype(int)
pred_test = (proba_test >= t_star_xgb).astype(int)
```

```
import pandas as pd

results = pd.DataFrame({
    "Split": ["Validation", "Test"],
    "ROC-AUC": [roc_auc_score(y_val, proba_val), roc_auc_score(y_test, proba_test)],
    "PR-AUC": [average_precision_score(y_val, proba_val), average_precision_score(y_test, proba_test)],
    "F1@t*": [f1_score(y_val, pred_val), f1_score(y_test, pred_test)],
    "Threshold (t*)": [t_star_xgb, t_star_xgb],
    "Best Trees": [best_ntree, best_ntree]
})

print(results)
```

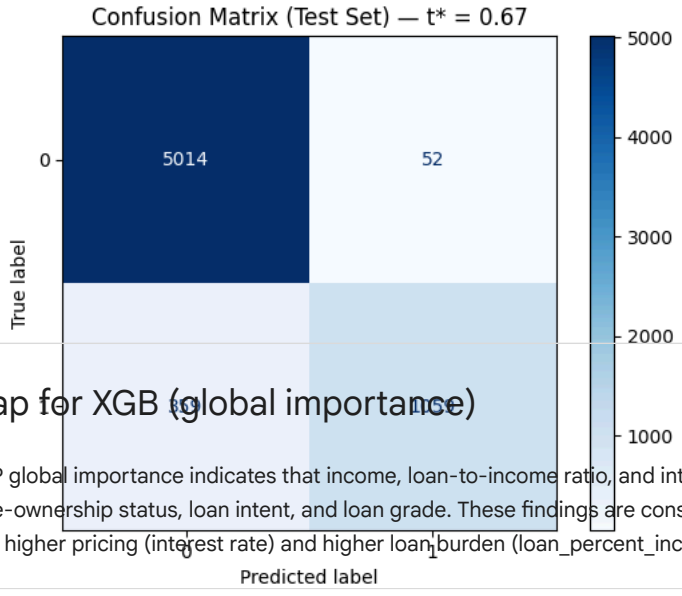
```
        Split   ROC-AUC    PR-AUC     F1@t*  Threshold (t*) Best Trees
0  Validation  0.947696  0.905223  0.836704            0.67       None
1        Test  0.946698  0.904489  0.837485            0.67       None
```

Draw PR curve and confusion matrix

```
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve, average_precision_score, precision_score, recall_score, f1_score

# Calculate the precision-recall curve from y_test and proba_test.
precision, recall, thresholds = precision_recall_curve(y_test, proba_test)
ap_score = average_precision_score(y_test, proba_test)

# Threshold-based label prediction t*
pred_test = (proba_test >= t_star_xgb).astype(int)

# Calculate F1 score, Precision, Recall at threshold t*.
f1 = f1_score(y_test, pred_test)
prec_t = precision_score(y_test, pred_test)
rec_t = recall_score(y_test, pred_test)

# Draw a PR curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, color='blue')
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title(f"Precision-Recall Curve (Test) – XGB\nAP = {ap_score:.4f}")

# Note below the chart
plt.figtext(0.5, -0.1,
            f"[XGB Test] F1 = {f1:.4f} | Precision = {prec_t:.4f} | Recall = {rec_t:.4f} | AP = {ap_score:.4f} | t* = {t_star_xgb:.
            wrap=True, horizontalalignment='center', fontsize=10)

plt.tight_layout()
plt.show()
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, pred_test)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title(f"Confusion Matrix (Test Set) – t* = {t_star_xgb:.2f}")
plt.show()
```



[XGB Test] F1 = 0.8375 | Precision = 0.9532 | Recall = 0.7468 | AP = 0.9045 | t* = 0.670



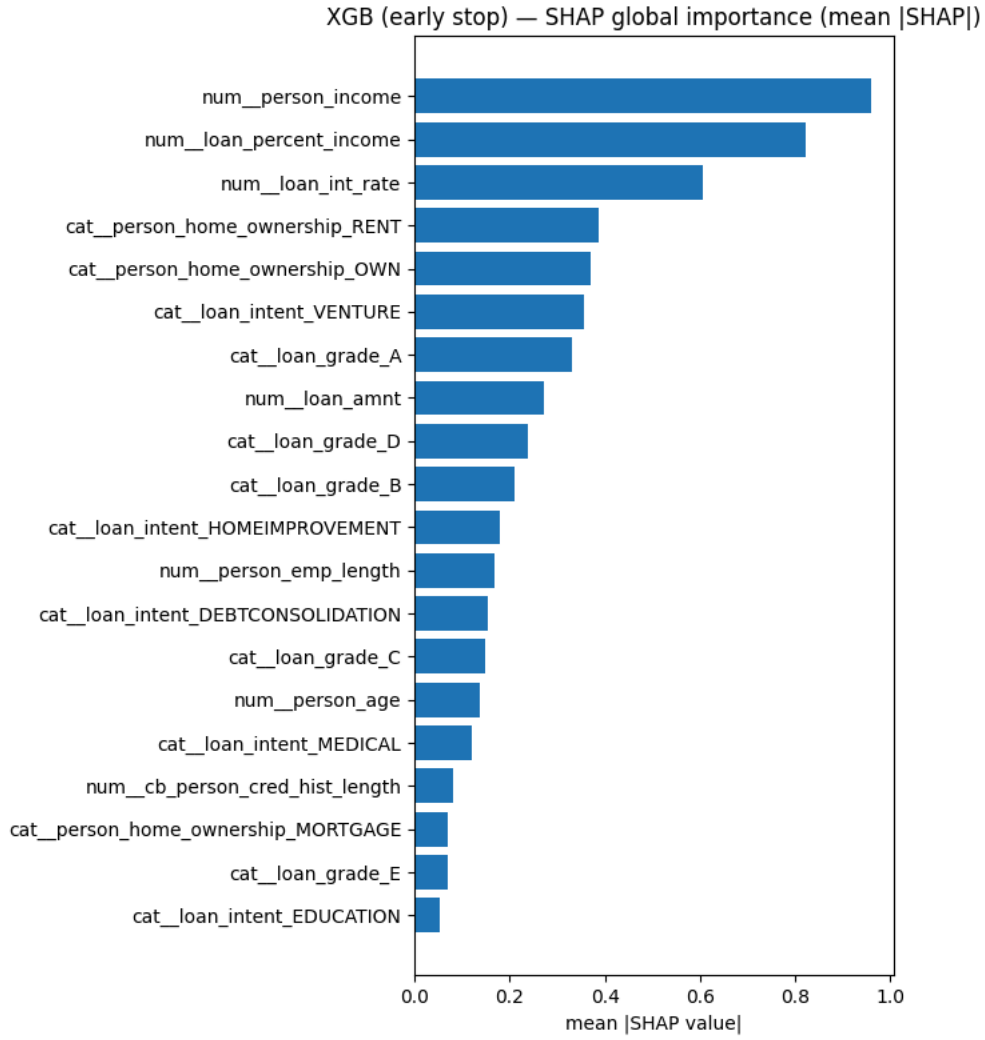
Shap for XGB (global importance)

SHAP global importance indicates that income, loan-to-income ratio, and interest rate are the dominant drivers of default risk, followed by home-ownership status, loan intent, and loan grade. These findings are consistent with credit-risk intuition: higher income lowers risk, while higher pricing (interest rate) and higher loan burden (loan_percent_income) increase risk.

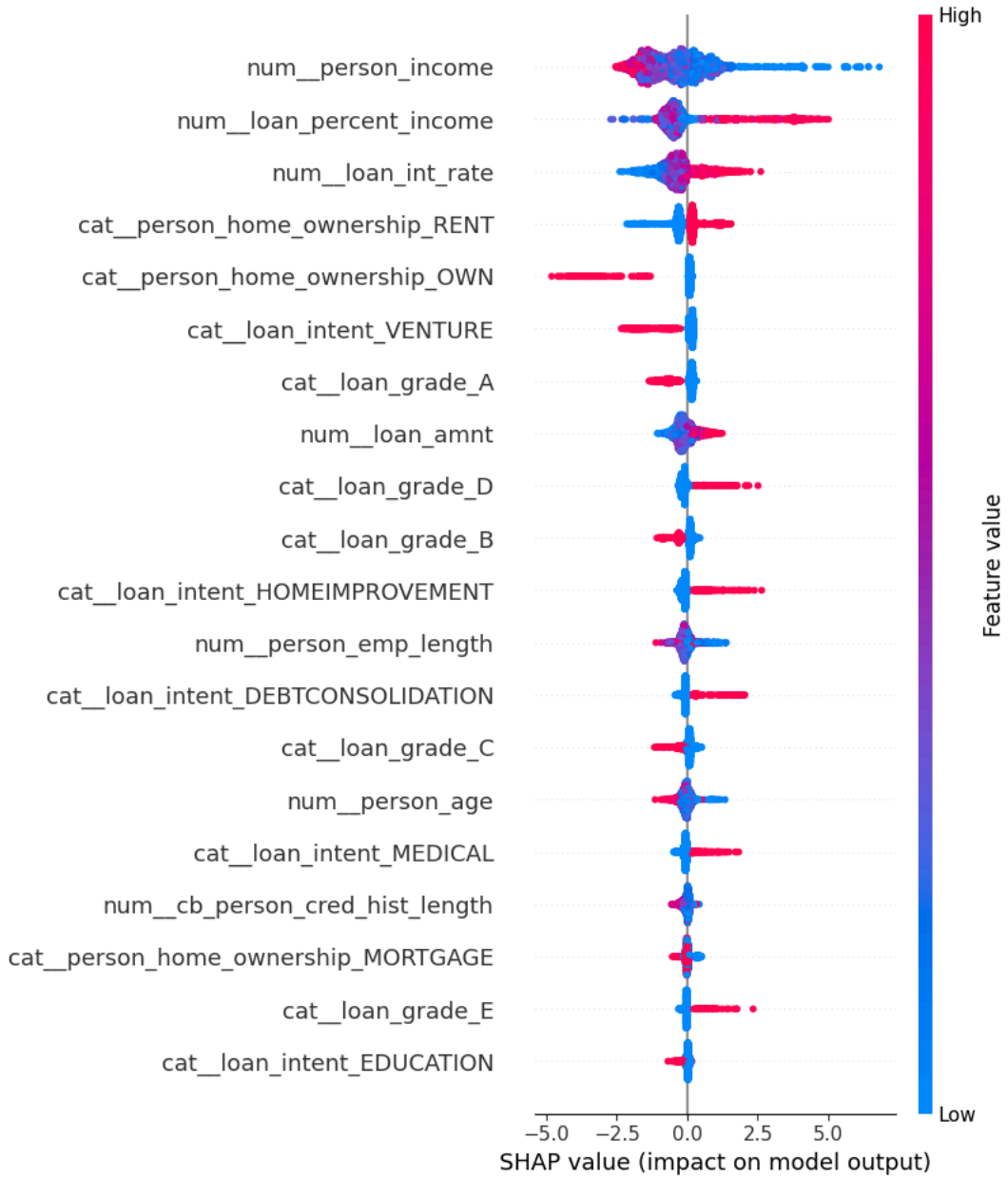
```
# === SHAP for XGB (global importance) ===
import numpy as np, pandas as pd, matplotlib.pyplot as plt, scipy.sparse as sp
import shap

# 1) Column name after preprocessing (OHE + numeric…)
feature_names = pre.get_feature_names_out()

# 2) Choose a background sample to calculate SHAP (subsample for speed).
Xbg = pre.transform(X_train)  # fit-on-TRAIN rồi transform
rng = np.random.RandomState(42)
bg_idx = rng.choice(Xbg.shape[0], size=min(2000, Xbg.shape[0]), replace=False)
Xbg_sub = Xbg[bg_idx]

# 3)Convert to dense if needed (SHAP tree supports dense well; 2000 rows are usually safe).
Xbg_dense = Xbg_sub.toarray() if sp.issparse(Xbg_sub) else np.asarray(Xbg_sub)

# 4) Calculate SHAP
```

```
explainer = shap.TreeExplainer(bst)         # BST is XGBoost. Booster from xgb.train.
shap_vals = explainer.shap_values(Xbg_dense)  # shape = (n_samples, n_features)

# 5) Global importance = mean(|SHAP|)
imp = np.abs(shap_vals).mean(axis=0)
imp_df = pd.DataFrame({"feature": feature_names, "mean_abs_shap": imp}) \
         .sort_values("mean_abs_shap", ascending=False)
print(imp_df.head(20))

# 6) Draw top-20
topk = 20
imp_top = imp_df.head(topk).iloc[::-1]
plt.figure(figsize=(7, 8))
plt.barh(imp_top["feature"], imp_top["mean_abs_shap"])
plt.title("XGB (early stop) — SHAP global importance (mean |SHAP|)")
plt.xlabel("mean |SHAP value|")
plt.tight_layout()
plt.show()
```

	feature	mean_abs_shap
6	num__person_income	0.958837
3	num__loan_percent_income	0.823162
2	num__loan_int_rate	0.605937
11	cat__person_home_ownership_RENT	0.386384
10	cat__person_home_ownership_OWN	0.370291
17	cat__loan_intent_VENTURE	0.355912
18	cat__loan_grade_A	0.330708
1	num__loan_amnt	0.270644
21	cat__loan_grade_D	0.239372
19	cat__loan_grade_B	0.208912
14	cat__loan_intent_HOMEIMPROVEMENT	0.178840
5	num__person_emp_length	0.168131
12	cat__loan_intent_DEBTCONSOLIDATION	0.152726
20	cat__loan_grade_C	0.147182
4	num__person_age	0.137367
15	cat__loan_intent_MEDICAL	0.118885
0	num__cb_person_cred_hist_length	0.080201
8	cat__person_home_ownership_MORTGAGE	0.070611
22	cat__loan_grade_E	0.070435
13	cat__loan_intent_EDUCATION	0.053979




```
shap.summary_plot(shap_vals, Xbg_dense, feature_names=feature_names, plot_type="dot")
```




```
print("best_iteration:", getattr(bst, "best_iteration", None))
print("best_score    :", getattr(bst, "best_score", None))
```

```
best_iteration: 473
best_score    : 0.9055037254150373
```

```
evals_result = {}
bst = xgb.train(params, dtr, num_boost_round=2000,
                evals=[(dtr,"train"), (dva,"val")],
                early_stopping_rounds=50,
                evals_result=evals_result,
                verbose_eval=False)
```

```
bst = xgb.train(params, dtr, 2000, evals=[(dtr,"train"), (dva,"val")],
                early_stopping_rounds=50, verbose_eval=100)
```

```
[0]     train-aucpr:0.83919     val-aucpr:0.83401
[100]   train-aucpr:0.90825     val-aucpr:0.88989
[200]   train-aucpr:0.94354     val-aucpr:0.90003
[300]   train-aucpr:0.96551     val-aucpr:0.90282
[400]   train-aucpr:0.97885     val-aucpr:0.90469
```

```
[500]   train-aucpr:0.98750     val-aucpr:0.90514
[522]   train-aucpr:0.98884     val-aucpr:0.90517
```

```
print("best_iteration:", bst.best_iteration)
print("best_val_aucpr:", bst.best_score)
# evals_result chứa lịch sử metric
train_curve = evals_result["train"]["aucpr"]
val_curve   = evals_result["val"]["aucpr"]
```

```
best_iteration: 473
best_val_aucpr: 0.9055037254150373
```

We trained XGB with n_estimators=2000, learning_rate=0.05 and early stopping (patience=50) on the Validation set using AUC-PR. Training stopped at 473 boosting rounds (Val AUC-PR 0.9055). Predictions on Val/Test use the best iteration only; the operating threshold t* is selected on Validation and then fixed for Test.

Artifact for the final model (Appendix)

```
# B1 — Save artifacts + self-check (lite)
import os, json, zipfile, joblib
import numpy as np, pandas as pd
from sklearn.metrics import average_precision_score, roc_auc_score, f1_score

os.makedirs("artifacts", exist_ok=True)

try:
    p_test = proba_test_xgb
except NameError:
    p_test = proba_test

# ===== 2) METRICS TEST FUNCTION (applied to VALIDATION) =====
metrics_xgb = {
    "PR-AUC": float(average_precision_score(y_test, p_test)),
    "ROC-AUC": float(roc_auc_score(y_test, p_test)),
    "F1@t":   float(f1_score(y_test, (p_test >= float(t_star_xgb)).astype(int))),
    "t":      float(t_star_xgb),
}

# ===== 3) Save ARTIFACTS =====
pd.Series(p_test, name="proba_xgb_test").to_csv("artifacts/proba_xgb_test.csv", index=False)
try:
    joblib.dump(xgb_model, "artifacts/xgb_model.pkl")
except NameError:

    try:
        bst.save_model("artifacts/xgb_model.json")
    except Exception:
        pass
with open("artifacts/metrics.json", "w") as f:
    json.dump({"XGB": {"TEST": metrics_xgb}}, f, indent=2)

with zipfile.ZipFile("artifacts.zip","w", zipfile.ZIP_DEFLATED) as z:
    for fn in os.listdir("artifacts"):
        z.write(os.path.join("artifacts", fn), arcname=fn)
print("✅ Saved artifacts.zip")

# ===== 4) SELF-CHECK =====
expected_xgb = {"PR-AUC": 0.9045, "F1@t": 0.8375}
assert abs(metrics_xgb["PR-AUC"] - expected_xgb["PR-AUC"]) <= 0.001
assert abs(metrics_xgb["F1@t"]   - expected_xgb["F1@t"])   <= 0.002
print("✅ Self-check passed ")
```

```
✅ Saved artifacts.zip
✅ Self-check passed
```

```
import json, pandas as pd, numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score, f1_score, confusion_matrix

# 1) Read artifacts
```